

SYLLABUS

Undecidability: Undecidability, A Language that is Not Recursively Enumerable, An Undecidable Problem That is RE, Undecidable Problems about Turing Machines.

LEARNING OBJECTIVES

- ✓ Definition of the Pushdown Automaton(PDA)
- ✓ The Languages of a PDA
- ✓ Equivalence of PDA's and CFG's
- ✓ Conversions of PDA to CFG and Vice versa
- ✓ Introduction to Turing Machine and its Formal Description
- ✓ Instantaneous Description and Language of Turing Machine
- ✓ Concept of Undecidability and Problems about Turing Machines
- ✓ Overview of Recursively Enumerable language
- ✓ A Language that is Not Recursively Enumerable
- ✓ An Undecidable Problem That is RE.

INTRODUCTION

finite automata (FA) is a special type of automata which is designed to recognize strings of symbols. It is denoted by $(Q, \Sigma, \delta, q_0, F)$ where Q is the set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state, and F is the set of final states.

A language L is said to be recursive if there exists a Turing machine (TM) denoted as M such that for every input symbol x from the alphabet Σ , M halts and either accepts or rejects x . If a language is recursive, in other words, solvable by a Turing machine, then it is called a recursive (or) algorithm. Whereas, a problem is said to be unsolvable if it is not solvable by a Turing machine. A language is called "recursive" if it is solvable by a Turing machine. These problems are easily solved by using algorithmic approach. However, a recursive language is a language defined over an input alphabet Σ in which a Turing machine performs the actions, like accept or reject. Moreover, a universal Turing machine is a special type of Turing machine that simulates other Turing machines by performing encoding process.

3.1 PUSH DOWN AUTOMATA

3.1.1 Definition of the Pushdown Automaton

Q1. Explain pushdown automata in detail. Also give example.

OR

Define Push Down Automata. Explain the basic structure of PDA with a neat graphical representation.

Answer :

Jan./Feb.-23(R18), Q6(a)

Pushdown Automata (PDA)

PDA can be defined as the automata or machine that processes all the context free languages. It is similar to the finite automata which is capable of processing only regular languages. It can be defined formally as well as informally.

Informal Definition

Finite automata cannot recognize every context free language and some context free languages are not regular. Therefore, some extra features are added to finite automata in order to accept any type of context free language. Finite automata with a stack memory can be viewed as a pushdown automata. The operation of stack is based on LIFO principle. This stack memory is also known as pushdown store which is capable of storing unlimited number of states.

The PDA performs state transmissions by observing symbols at top of the stack, present state and on the input symbol. A part from this, it may also perform spontaneous transition through ϵ rather than input symbol. The PDA performs the following operations in a single transition.

(a) It utilizes input symbols within the transition. In case of ϵ usage, it does not use input symbols.

(b) It changes the state which is not dependent on the input symbol.

(c) It pushes the symbol present at the top of the stack by any string. The different strings produce different effects on stack as listed below,

- (i) ϵ String: This string pops off the content present on top of the stack.
- (ii) Symbol Same as Previous One: It does nothing to the stack.
- (iii) Different Symbol: It changes the top of the stack but without push or pop operation.
- (iv) Multiple Symbols: It changes the top of the stack with one or more push operations.

Model of PDA

The model of push down automata is shown below.

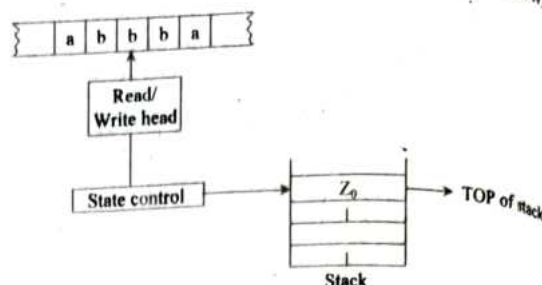


Figure: Model of PDA

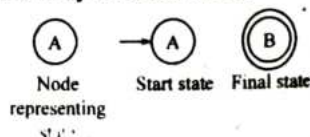
The model of push down automata consists of,

- (i) Input tape
 - (ii) Read write head
 - (iii) State controller
 - (iv) Stack.
- (i) **Input Tape**
It is a sequential tape that consists of finite number of cells where each cell holds a single input symbol.
- (ii) **Read Write Head**
It performs read/write operation on cells of input type.
- (iii) **State Controller**
It takes decision regarding which input symbol to be read next.
- (iv) **Stack**
It is used for storing temporary items and for performing PUSH and POP operations.

Graphical Representation of PDA

A pushdown automata can be represented graphically in the form of transition diagram. This transition diagram is a directed graph that shows diagram of finite automata.

1. A state of the PDA is called the start state of transition diagram. It is denoted as the start state containing arrow labelled arc in order to represent the start state. The last/final state of the transition diagram usually called accepting state is denoted by a double circle.



2. A labelled arc connects two states between each state.
3. A labelled arc between each state of form $0, X/\alpha$ specifies the input used and the old and new elements on top of the stack.

Formal Definition

Formally, a pushdown automata is defined as a seven tuple machine $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.

Where,

Q = Set of states (finite)

Σ = Set of input symbols

Γ = Set of symbols to push on the stack

δ = Transition function mapping from $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow (Q, \Gamma)$

q_0 = Initial state.

$F \subseteq Q$ is the set of final states.

Example

Consider the generation of PDA for a palindrome over the alphabet $\{0, 1\}$. To do so, consider PDA,

$$P = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$$

Where,

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{0, Z_0\}$$

δ = Transition

$$q_0 = q_0$$

$$Z_0 = Z_0$$

$$F = q_2$$

The transition function δ is given by,

$$\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$$

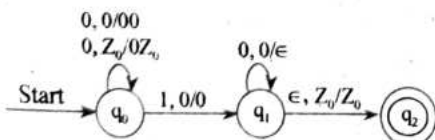
$$\delta(q_0, 0, 0) = \{(q_0, 00)\}$$

$$\delta(q_0, 1, 0) = \{(q_1, 0)\}$$

$$\delta(q_1, 0, 0) = \{(q_1, \epsilon)\}$$

$$\delta(q_1, \epsilon, Z_0) = \{(q_2, \epsilon)\}$$

The transition diagram for the PDA is shown below,



The possible moves of strings 00100 and 00101 with the help of computations tree is shown below,

$$(q_0, 00100, Z_0) \vdash (q_0, 0100, 0Z_0)$$

$$\vdash (q_0, 100, 00Z_0)$$

$$\vdash (q_1, 00, 00Z_0)$$

$$\vdash (q_1, 0, 0Z_0)$$

$$\vdash (q_1, \epsilon, Z_0)$$

$$\vdash (q_2, \epsilon, Z_0) \text{ (accept)}$$

$\therefore$ The string 00100 is accepted by the above PDA. Therefore the given string is a palindrome.

Now check the acceptance of string $w = 00101$

$$(q_0, 00101, Z_0) \vdash (q_0, 0101, 0Z_0)$$

$$\vdash (q_0, 101, 00Z_0)$$

$$\vdash (q_1, 01, 00Z_0)$$

$$\vdash (q_1, 1, 0Z_0)$$

As the transition on $(q_1, 1, 0)$ is not defined by the above PDA, the given string cannot be processed further. Therefore the given string $w = 00101$ is rejected by the above PDA. Hence the string $w = 00101$ is not a palindrome.

Q2. What do you mean by an instantaneous description of a PDA? Explain with example.

Answer :

Instantaneous Description

An instantaneous description (ID) of a push down automata $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is written as (q, x, z) where x is the input string in Σ^* and z is the input string in Γ^* .

Example

Consider an ID $(q, x_1, x_2, x_3, \dots, x_m, z_1, z_2, z_3, \dots, z_n)$. This describes the PDA that the input string $x_1, x_2, x_3, \dots, x_m$ should be processed from the current state in the same order. The stack which is called as push down store (PDS) contains $z_1, z_2, z_3, \dots, z_n$ with z_1 at the top and z_n at the bottom.

Initially, the ID is (q_0, x, z_0) where q_0 is initial state, x is the input string to be processed and z_0 is the only symbol on the PDA.

The change in IDs is described by the move relation which is denoted by $\vdash$. The move relation between ID's is as follows:

$$(q, x_1 x_2 \dots x_m, z_1 z_2 \dots z_n) \vdash (q', x_2 x_3 \dots x_m, \epsilon z_2 z_3 \dots z_n)$$

If (q, x_1, z_1) contains (q', α)

The move relation is illustrated in the figure below.

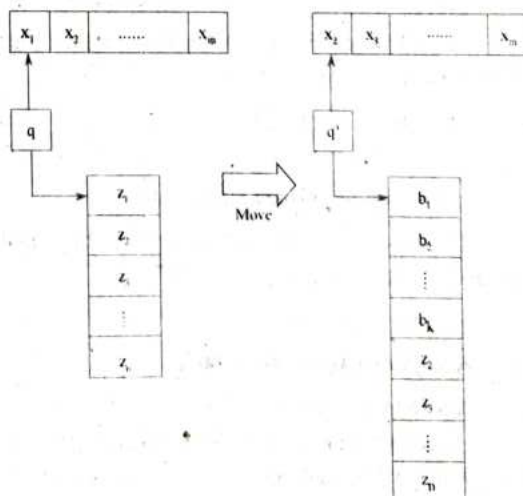


Figure: Move Relation

Example

Consider the push down automata for language
 $L = \{a^{2n} b^{n/n} \geq 1\}$

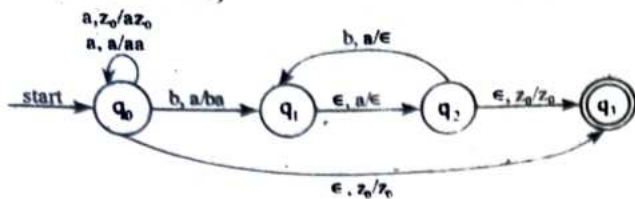


Figure: PDA for Language $L = \{a^{2n} b^{n/n} \geq 1\}$

Consider an input string $W = aab$. The instantaneous description of string $W = aab$ is as follows,

$$\begin{aligned} (q_0, aab, z_0) &\vdash (q_0, aab, z_0) \\ &\vdash (q_0, ab, az_0) \\ &\vdash (q_0, b, aaz_0) \\ &\vdash (q_1, \epsilon, az_0) \\ &\vdash (q_2, \epsilon, z_0) \\ &\vdash (q_3, \epsilon, z_0) = (\text{Accept}) \end{aligned}$$

Since the final state is reached, the string $W = aab$ is accepted by PDA.

3.1.2 The Languages of a PDA

Q3. How does the language is accepted by PDA? Explain with examples.

Answer :

Acceptance of Language by PDA

A PDA accepts a language in two ways,

- (i) Acceptance by final state and
- (ii) Acceptance by empty stack.

(i) Acceptance by Final State

A pushdown automata is defined by, $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ accepts a given language by generating the strings $w \in \Sigma^*$, by entering the final state such that,

$$(q_0, w, Z_0) \vdash (q_f, \epsilon, \alpha) \text{ where } q_f \in F \text{ and } \alpha \in \Gamma^*$$

i.e., A PDA can accept strings through final state based on the following conditions,

- (a) If strings is completely traversed (i.e., it comes to an end).
- (b) If the PDA has visited its final state.

Thus if a strings of PDA are accepted by final state then the final state, should be of type,

$$\delta(q_f, \epsilon, Z_0) = (q_f, Z_0)$$

(ii) Acceptance by Empty Stack

A pushdown automata is defined by, $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ accepts a given language by generating the set of strings $w \in \Sigma^*$ by making its stack empty such that,
 $(q_0, w, Z_0) \vdash (q_f, \epsilon, \epsilon)$

A PDA can accept strings by empty stack based on the following conditions,

- (a) If string is completely traversed (i.e., it comes to an end).
- (b) If the PDA has its stack empty.

Thus, if strings of PDA are accepted by empty stack then the final state should be of type,

$$\delta(q_f, \epsilon, Z_0) \vdash (q_f, \epsilon)$$

Q4. Design a PDA for $L = \{a^i b^j c^k \mid j \geq i + k \text{ and } ijk > 0\}$. Process the string aaabbbbbbcc using instantaneous description.

OR

Construct a PDA which accepts language of word over alphabet $\{a, b\}$ canting $\{a^i b^j c^k \mid i, j, k \in \mathbb{N}, i + k = j\}$.

Answer :

Jan/Feb-23(R18), Q7(a)

Given that,

$$L = \{a^i b^j c^k \mid j \geq i + k\} \text{ and } ijk > 0$$

The PDA 'M' that accepts the language 'L' is given below,

$$M = \{q_0, q_1, q_2\}, \{a, b, c\}, \{z_0, a, b, c\}, \{\delta, q_0, z_0, q_2\}$$

Where 'δ' is given by,

$$\delta(q_0, a, z_0) = \{q_0, az_0\}$$

$$\delta(q_0, a, a) = \{q_0, aa\}$$

$$\delta(q_0, b, a) = \{q_1, \epsilon\}$$

$$\delta(q_1, b, z_0) = \{q_1, cz_0\}$$

$$\delta(q_1, b, c) = \{q_1, cc\}$$

$$\delta(q_1, c, c) = \{q_2, \epsilon\}$$

$$\delta(q_2, c, c) = \{q_2, \epsilon\}$$

$$\delta(q_2, \epsilon, z_0) = \{q_2, \epsilon\}$$

$$\delta(q_2, \epsilon, c) = \{q_2, \epsilon\}$$

$$\delta(q_2, \epsilon, \epsilon) = \{q_2, \epsilon\}$$

The pushdown automata for the given language is as follows,

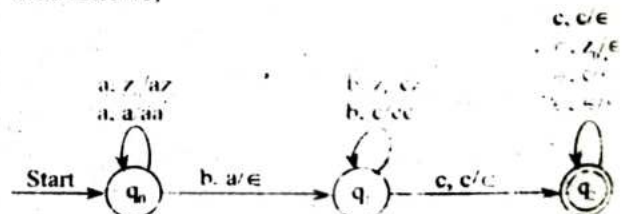


Figure: PDA for the Language L

Processing the string aaabbbbbbcc on PDA.

$$\begin{aligned} \delta(q_0, aaabbbbbbcc, z_0) &\vdash \delta(q_0, aabbbbbbcc, az_0) \\ &\vdash \delta(q_0, abbbbbbcc, aaaz_0) \\ &\vdash \delta(q_1, bbbbbbcc, aaaz_0) \\ &\vdash \delta(q_1, hbbbbbcc, aaaz_0) \\ &\vdash \delta(q_1, hbbbbbcc, az_0) \end{aligned}$$

$\vdash \delta(q_1, bbbcc, z_0)$
 $\vdash \delta(q_1, bbcc, cz_0)$
 $\vdash \delta(q_1, bcc, ccz_0)$
 $\vdash \delta(q_1, cc, cccz_0)$
 $\vdash \delta(q_1, c, cccc_0)$
 $\vdash \delta(q_1, \epsilon, cz_0)$
 $\vdash \delta(q_1, \epsilon, z_0)$
 $\vdash \delta(q_1, \epsilon, \epsilon) \dots \dots$ Accept

Thus, the string **aaabbbbcc** is accepted by PDA.

Q5. Design a Push Down Automata to accept $\{0^n 1^n \mid n > 1\}$. Draw the transition diagram for the PDA. Show by instantaneous description that the PDA accepts the string '0011'.

Answer :

Given that $L = \{0^n 1^n \mid n > 1\}$

The language contains equal number of 0's followed by equal number of 1's i.e $L = \{0011, 000111, 00001111, \dots\}$

Let PDA be $M = (\{q_0, q_1, q_2\}, \{0, 1\}, \{Z_0, 0, 1\}, \{q_0, Z_0, q_1\})$

The instantaneous description is given below.

$R_1: \delta(q_0, 0, Z_0) = (q_0, 0Z_0)$

$R_2: \delta(q_0, 0, 0) = (q_0, 00)$

$R_3: \delta(q_0, 1, 0) = (q_1, \epsilon)$

$R_4: \delta(q_1, 1, 0) = (q_1, \epsilon)$

$R_5: \delta(q_1, \epsilon, Z_0) = (q_2, \epsilon)$

Where q_0 is starting state and q_2 is final state.

The transition diagram for above PDA is as follows.

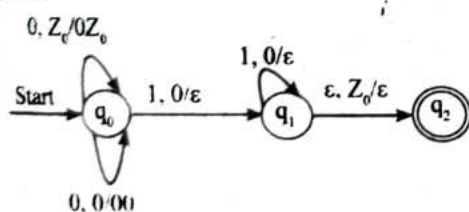


Figure: PDA for Language $\{0^n 1^n \mid n > 1\}$

Given string is 0011

The simulation of the string is as follows,

$\delta(q_0, 0011, Z_0) \vdash \delta(q_0, 011, 0Z_0) \quad [\because R_1]$

$\vdash \delta(q_0, 11, 00Z_0) \quad [\because R_2]$

$\vdash \delta(q_1, 1, 0Z_0) \quad [\because R_3]$

$\vdash \delta(q_1, \epsilon, Z_0) \quad [\because R_4]$

$\vdash \delta(q_2, \epsilon) \quad [\because R_5]$

(Accepted)

Since final state is reached the given string 0011 is accepted by the above PDA.

Q6. Construct a PDA that accepts the language, $L \{wcw^R \mid w \in (a+b)^*\}$

Answer :

Dec.-19(R16), Q6

Given that,

$L = \{wcw^R \mid w \in (a+b)^*\}$

The given language accepts the strings of palindrome including the prefix c in between.

Example

The given language accepts strings like $L = \{01c10, 10c01, 0110c0110, \dots\}$

Let the PDA be $P = (\{q_0, q_1\}, \{a, b\}, \{x, z_0\}, \delta, q_1, z)$

Where δ is,

$R_1: \delta(q_0, a, z_0) = \delta(q_0, az_0)$

$R_2: \delta(q_0, b, z_0) = \delta(q_0, bz_0)$

$R_3: \delta(q_0, a, a) = \delta(q_0, aa)$

$R_4: \delta(q_0, a, b) = \delta(q_0, ab)$

$R_5: \delta(q_0, b, a) = \delta(q_0, ba)$

$R_6: \delta(q_0, b, b) = \delta(q_0, bb)$

$R_7: \delta(q_0, c, z_0) = \delta(q_1, z_0)$

$R_8: \delta(q_0, c, a) = \delta(q_1, a)$

$R_9: \delta(q_0, c, b) = \delta(q_1, b)$

$R_{10}: \delta(q_1, a, a) = \delta(q_1, \epsilon)$

$R_{11}: \delta(q_1, b, b) = \delta(q_1, \epsilon)$

$R_{12}: \delta(q_1, \epsilon, z_0) = \delta(q_1, \epsilon)$

The transition diagram for the above PDA is shown below,

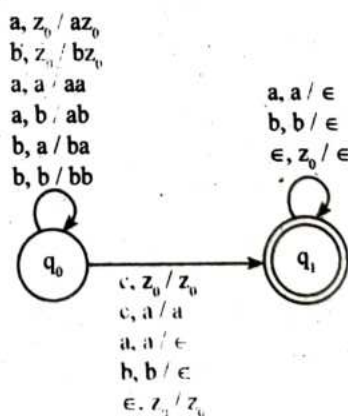


Figure: Transition Diagram for PDA

The processing of string $w = abba$ is shown below,

$(q_0, abba, z_0) \vdash (q_0, \underline{b}ba, \underline{a}z_0) \quad [\because R_1]$

$\vdash (q_0, \underline{b}a, \underline{ba}z_0) \quad [\because R_5]$

$\vdash (q_0, \underline{a}ba, \underline{bb}az_0) \quad [\because R_6]$

$\vdash (q_0, \underline{a}bba, \underline{abb}az_0) \quad [\because R_4]$

$\vdash (q_1, \underline{a}bba, \underline{abb}az_0) \quad [\because R_8]$

$\vdash (q_1, bba, bba z_0)$	$[\because R_{10}]$
$\vdash (q_1, ba, baz_0)$	$[\because R_{11}]$
$\vdash (q_1, a, az_0)$	$[\because R_{11}]$
$\vdash (q_1, \epsilon, z_0)$	$[\because R_{10}]$
$\vdash (q_1, \epsilon, z_0)$	$[\because R_{12}]$

$\therefore$ The string 0110c0110 is accepted by the above PDA.

Q7. Design pushdown automata for $L = \{a^{2n}b^n \mid n \geq 1\}$.

Answer :

May-19(R16), Q6(b)

Given language is,

$$L = \{a^{2n}b^n \mid n \geq 1\}$$

The given language accepts the set of strings containing even number of a's followed by any number of b's (or) twice the number of a's than b's.

Let the PDA be $M = \{(q_0, q_1, q_2, q_3), \{a, b\}, \{z_0, a, b\}, \{q_0, z_0, q_1\}\}$

Where δ is given as,

$$(q_0, a, z_0) = (q_0, a, z_0)$$

$$(q_0, a, a) = (q_0, aa)$$

$$(q_0, \epsilon, z_0) = (q_3, \epsilon)$$

$$(q_0, b, a) = (q_1, \epsilon)$$

$$(q_1, \epsilon, a) = (q_2, \epsilon)$$

$$(q_1, b, a) = (q_2, \epsilon)$$

$$(q_2, b, a) = (q_1, \epsilon)$$

$$(q_2, \epsilon, a) = (q_1, \epsilon)$$

$$(q_2, \epsilon, z_0) = (q_3, \epsilon)$$

The transition diagram (PDA) is as follows,

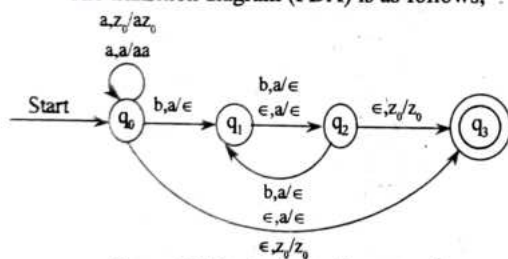


Figure: PDA for Language $\{a^{2n}b^n\}$

The processing of string $aaaabb$ is shown below,

$(q_0, aaaabb, z_0)$	$(q_0, aaabb, az_0)$
	$(q_0, aabb, aaz_0)$
	$(q_0, abb, aaaz_0)$
	$(q_0, bb, aaaaaz_0)$
	$(q_1, b, aaaz_0)$
	$(q_2, \epsilon, aaaz_0)$
	(q_1, ϵ, aaz_0)
	(q_2, ϵ, az_0)
	(q_3, ϵ, z_0)
	(q_3, ϵ)

Since, final state is reached the string $aaaabb$ is accepted by above PDA.

3.1.3 Equivalence of PDA and CFG's, Acceptance by Final State

Q8. Discuss the equivalence between PDA and CFG/CFL.

Answer :

Equivalence of PDA's and CFG's/CFL

Equivalence of PDA's and CFG's can be proved based on three assumptions of languages which are similar to each other. If ' L ' is a language then three classes of languages can be specified as follows,

1. It must be defined by CFG's.
2. It must be accepted by final state of PDA.
3. It must be accepted by empty stack of PDA.

Since, it is already proved that (2) and (3) are similar to each other. So, it can be proved that (1) and (3) are also similar to each other by indicating/intimating the equivalence of these languages.

The CFL's can be defined in three different ways by grouping constructions of PDA which is shown below.

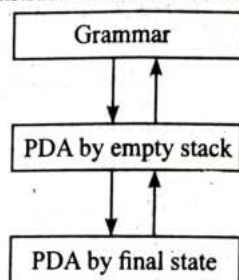


Figure (1): The Different Ways for Defining CFL's by Grouping Constructions of PDA

The grammar ' G ' can be converted to PDA ' P ' as follows,

The CFG ' G ' is given as follows,

$$G = (V, T, P, S)$$

Where,

V = Set of non-terminals

T = Set of terminals

P = Productions

S = Start symbol

Now, constructing a PDA ' P ' for given CFG in such a way that it simulates the leftmost derivation of CFG i.e. G . The left-sentential form of a non-terminals are written as ' $\alpha X \alpha$ ' whereas, the left-sentential form of a terminals are written as ϵ .

Here,

' X ' indicates the left most variable.

' a ' indicates the terminals that occur to the left of ' X '.

' α ' indicates the string of variables and terminals that occur to the right of ' X '.

Where, α and ϵ are referred as the tail of left-sentential form.

The construction of PDA is done by simulating the series of left-sentential form in order to produce the given terminal string (w). During this construction the tail of left-sentential form of the production ' $aX\alpha$ ' is placed on the stack in such a way that the non-terminal ' X ' appears at the top of the stack, and the terminal ' a ' appears on the input tape.

If ' w ' is an input string, then first input should be ' a ' followed by any number of inputs.

Consider, $w = 'ab'$ is an input string in which ' a ' is already present on the input tape and ' b ' should be placed in such a manner that, it should be a prefix of ' a ' on the input tape.

In PDA, the instantaneous description (ID) of left-sentential form of ' $aX\alpha$ ' is represented as $\delta(q_0, b, \beta\alpha)$. If $X \rightarrow \beta$ then ' β ' should be placed at the top of the stack rather than ' X ' and ID becomes $\delta(q_0, b, \beta\alpha)$. Where, ' q_0 ' is the only single state present in this PDA.

Now, $\delta(q_0, b, \beta\alpha)$ cannot be used for representation of next left-sentential form, since ' β ' contains prefix of terminals or literally zero variables and α contains prefix of variables.

In the left-sentential form, if the terminals are present before the variable then they should be deleted in order to show the upcoming variable at the top of the stack.

However, if the terminals are matched across the next input symbol, then it ensures that the assumptions made by us at the leftmost derivation are correct and gives result as success. If the assumptions are incorrect then it gives result as fail (i.e., PDA is dropped off).

If the result is success then it gives correct left-sentential form of ' w '. At this point if the symbols present at the top of the stack are variables then they can be expanded. Else, if they are terminals then they are matched across each and every input. So, it can be said that, the stack is empty and it is known as "Empty stack".

The informal construction of PDA can be done in the following manner,

Let the CFG be $G = (V, T, P, S)$

Where, V = Set of non-terminals

T = Set of terminals

P = Productions

S = Start symbol

And PDA be $P = (\{q\}, T, V, \delta, q, S, \phi)$

Where, $q = \{q_0, q_1, q_2, \dots, q_p\}$

T = Set of terminals

V = Set of non-terminals

$q = q_0$

S = Start symbol

$\phi = q_f$

δ is defined as follows,

- For every individual variable X ,
 $\delta(q_0, \epsilon, X) = \{(q_0, \beta) \mid X \rightarrow \beta \text{ is a production of } P\}$
- For every individual terminal ' a '
 $\delta(q_0, \epsilon, a) = \{(q_0, \epsilon)\}$

The PDA ' P ' can be converted to Grammar ' G ' as follows,

The PDA ' P ' is given as follows, $P = \{Q, \Sigma, \Gamma, \delta, q_0, F\}$

Where,

Q = Set of states

Σ = Set of input symbols

Γ = Set of symbols to push on the stack

δ = Transition function

q_0 = Initial state

F = Final state.

Now, constructing a CFG ' G ' for a given PDA ' P ' in such a way that its language should be similar to the language of PDA which is accepted by empty stack. The concept behind this is that, conversion is done in order to check whether the CFG and PDA are equivalent or not and also to identify fundamental events. These events are the events that are present in the history of PDA's. They are used for processing the given input by popping of single symbol from the stack and by consuming its input.

When symbols are popped out of the stack then its stack changes from some state q_0 to another state q_1 .

Consider the following,

- Symbols - $B_1, B_2, B_3, \dots, B_n$
- Inputs - $a_1, a_2, a_3, \dots, a_n$ and
- States - $q_0, q_1, q_2, \dots, q_p$

If the symbol B_1 is popped out of the stack then its state change from q_0 to q_1 by reading its input a_1 which is shown in figure below.

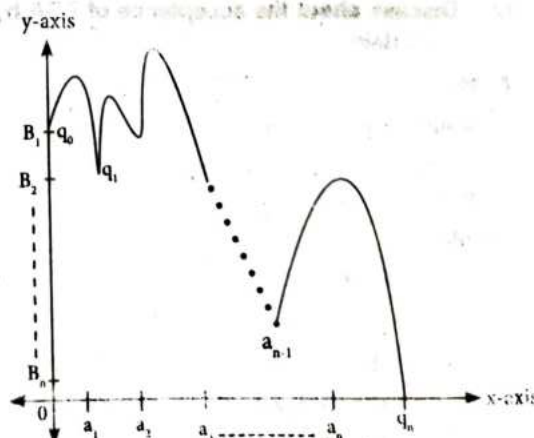


Figure (2): When PDA Makes a Move by Popping the Symbol Out of the Stack

Similarly, this procedure will continue until all the symbols are popped from the stack by reading its corresponding input.

To construct an equivalent grammar 'G' set of variables defining an "event" are required. They includes the following things.

- Symbols like 'X' popped from the stack.
- After symbol 'X' is popped i.e., eventually replaced by ε changed from previous state i.e., q_0 to next state i.e., q_1 .

Thus, the representation of the variables are done by making use of composite symbol like $[q_0 X q_1]$. Here, the variable 'X' is described by making use of series of characters. (i.e., q_0 and q_1).

Let the PDA be $P = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$ and CFG be $G = (V, T, P, S)$.

Where,

$V = \{S, q, q', Z\}$ such that q and $q' \in Q$ and $Z_0 \in \Gamma$

$T = \{0, 1\}$

$S = S$

The production rules (P) of CFG (G) for the PDA P can be constructed based on the following three rules.

- The production rule for start symbols can be of form $S \rightarrow [q_0, Z_0, q_1]$.
- The production rule of CFG for PDA move can be of form $\delta(q, 0, Z) = \{(q', \epsilon)\}$ is given by,
- The production rule of CFG for PDA moves of form

$$\delta(q, Z, Z') \rightarrow 0$$

$$\delta(q, 0, Z) = \{(q', Z_1, Z_2, Z_3, \dots, Z_n)\}$$

can be written as,

$$\delta(q, 0, Z) \rightarrow 0[q, Z_1, q_2] [q_2, Z_2, Z_3] [q_3, Z_3, Z_4] [q_4, Z_4, Z_5] [q_5, Z_5, Z_6], \dots$$

Q9. Discuss about the acceptance of PDA by final state.

Answer:

Acceptance by Final State

For answer refer Unit-III, Page No. 68, Q3, Topic: Acceptance by Final State.

Example

Let PDA be $M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$

$$Q = \{S, A, B\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{a, b, Z_0\}$$

$$F = \{B\}$$

Assume, language $L = \{ww^R \mid w \in \Sigma^+\}$

The transitions for the above language are as follows.

$$R_1: \delta(S, a, Z_0) = (S, aZ_0)$$

$$R_2: \delta(S, b, Z_0) = (S, bZ_0)$$

$$R_3: \delta(S, a, a) = (S, aa)$$

$$R_4: \delta(S, b, b) = (S, \epsilon)$$

$$R_5: \delta(S, a, b) = (S, ab)$$

$$R_6: \delta(S, b, a) = (S, ba)$$

$$R_7: \delta(S, \epsilon, a) = (A, a)$$

$$R_8: \delta(S, \epsilon, b) = (A, b)$$

$$R_9: \delta(A, a, a) = (A, \epsilon)$$

$$R_{10}: \delta(A, b, b) = (A, \epsilon)$$

$$R_{11}: \delta(A, \epsilon, Z_0) = (B, Z_0)$$

The transition diagram for the above language is as follows,

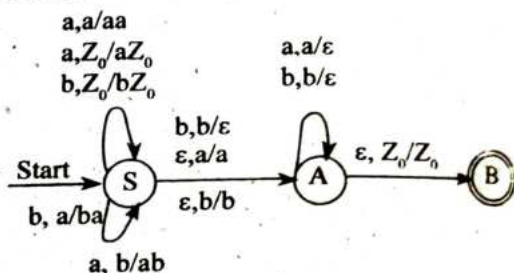


Figure: Pushdown Automata

The pushdown automata works as follows,

- Initially, the machine M remains in its initial state 'S' as long as it completes reading half of its input. It uses transitions from 1 to 3 and 5, 6 to push the input onto the stack.
- In state B , the machine ' M ' pops the topmost symbol of the stack, iff it matches the next symbol, otherwise the machine T halts.
- If machine reads the configuration (A, ϵ, Z_0) the final state with end of input and empty stack then machine accepts the input.

Now, check the acceptance

of S string

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

$S \rightarrow aabbaa$

Since B is the final state, the string $aabbaa$ is accepted by the constructed PDA.

Q10. Prove that acceptance by empty stack and by final state is equivalent.

Answer :

Let, $P = (Q, \Sigma, \tau, \delta, q_0, z_0, F)$ accepts a language (say lang) by considering its final state. Here, PDA Q should be determined which accepts lang by means of empty stack. This PDA obtained should remain equivalent to the PDA by considering the final state i.e., in terms of an equation. This scenario can be expressed as follows.

$$\text{Lang} = I(P) = J(Q)$$

PDA obtained by means of final state

PDA obtained by means of empty stack

To prove the above consequence, consider three major aspects.

1. When initial movement of Q is done then the initial identifier of P is reached.
2. Once the above condition is satisfied, P resembles Q till final state of Q is reached.
3. Once the condition 2 is satisfied, two things can happen, either Q simulates P or P empty its stack depending on the input string.

Now.

$$Q = (S \cup \{\bar{q}_0, d\}, \Sigma, \tau \cup \{z_0^1\}, \delta_Q, \bar{q}_0, \bar{z}_0, f)$$

$$\delta_Q(q_0, \epsilon, z_0) = \{(q_0, z_0, z_0')\} \quad \dots \text{Rule (1)}$$

$$\delta_Q(q, a, z) = \{(q, a, z)\} \quad \dots \text{Rule (2)}$$

$$\delta_Q(q, \epsilon, z) = \delta\{(q, \epsilon, z) \cup (d, \epsilon)\} \quad \dots \text{Rule (3)}$$

$$\delta_Q(d, \epsilon, z) = \{(d, \epsilon)\} \quad \dots \text{Rule (4)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha)$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

$$(q_0, w, z_0) \vdash^* (q, \epsilon, \alpha) \quad \text{Rule (2)}$$

With this, one can conclude that "acceptance by final state and empty stack is equal".

Q11. Construct a PDA to accept the following language $L = \{a^n b^n | n > 0\}$.

Answer :

Sep.-21(R18), Q6(a)

Given that,

$$L = \{a^n b^n | n \geq 1\}$$

The language contains equal number of a 's followed by equal number of b 's.

Let, the PDA be

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, Z_0, q_2)$$

Assume that, the stack symbol is Z_1 . Whenever 'a' is traversed from the input tape, ' Z_1 ' is pushed into the stack and δ is,

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

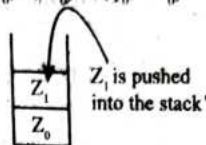


Figure (1): Stack

If the input 'a' is traversed more than once then the transition occurs from state q_0 to itself (i.e., to q_0 only) and δ is,

$$\delta(q_0, a, a) = (q_0, aa)$$

If the input 'b' is traversed from the input tape then Z_1 is popped from the stack and state changes from q_0 to state q_1 and δ is,

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

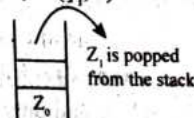


Figure (2): Stack

If the input 'b' is traversed more than once, then the transition occurs from state q_1 to itself (i.e., to q_1 only) and δ is,

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

Now, the PDA (M) is in state q_1 with stack symbol Z_1 .

If M is designed with empty stack then δ is defined as,

$$\delta(q_1, \epsilon, Z_0) = (q_2, \epsilon)$$

If M is designed with final state then δ is defined as,

$$\delta(q_1, \epsilon, Z_0) = (q_2, Z_0)$$

Now δ is defined as follows.

$$R_1: \delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$R_2: \delta(q_0, a, a) = (q_0, aa)$$

$$R_3: \delta(q_0, b, a) = (q_1, \epsilon)$$

$$R_4: \delta(q_1, b, a) = (q_1, \epsilon)$$

$$R_5: \delta(q_1, \epsilon, Z_0) = (q_2, \epsilon) \text{ is accepted by empty stack.}$$

$$R_6: \delta(q_1, \epsilon, Z_0) = (q_2, Z_0) \text{ is accepted by final state.}$$

Here, ' Z_0 ' is the stack bottom symbol.

The transition diagram for 'M' is shown below,

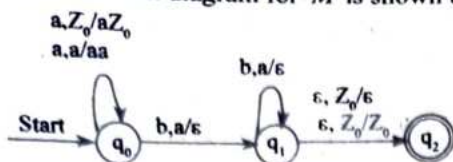


Figure (3): Transition Diagram for 'M'

Consider a string of equal number of *a*'s and *b*'s and simulate the above PDA for the string.

Let string be $w = aabb$.

$$\begin{aligned}
 (q_0, aabb, Z_0) &\vdash (q_0, abb, aZ_0) & [\because R_1] \\
 &\vdash (q_0, bb, aaZ_0) & [\because R_2] \\
 &\vdash (q_1, b, aZ_0) & [\because R_3] \\
 &\vdash (q_1, \epsilon, Z_0) & [\because R_4] \\
 &\vdash (q_2, \epsilon) & [\because R_5] \\
 &[\text{Acceptance by empty stack}]
 \end{aligned}$$

(or)

$$\begin{aligned}
 &\vdash (q_2, Z_0) & [\because R_6] \\
 &[\text{Acceptance by final state}]
 \end{aligned}$$

Since final state is reached the string containing equal number of *a*'s and *b*'s are accepted by the empty stack and final state of the above constructed PDA.

Q12. Prove that if PDA *P* is constructed from CFG *G*, then $N(P) = L(G)$.

Answer :

To show that $N(P) = L(G)$, it must be proved that '*w*' should be in $N(P)$ if and only if the input '*w*' is in $L(G)$.

To prove this theorem,

Consider an input '*w*' in $L(G)$ with leftmost derivation of form $Z = \gamma_1 \xRightarrow{lm} \gamma_2 \xRightarrow{lm} \dots \xRightarrow{lm} \gamma_n = w$

Then,

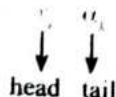
Take an induction step of '*K*' such that,

$$(q, w, Z) \xRightarrow{*}_P (q, \gamma_k, \alpha_k) \quad \dots (1)$$

Where,

1. γ_k and α_k are the symbols for representing left sentential form of γ_k
2. γ_k represents the tail of γ_k and

Thus γ_k will be



4. In $x_i \gamma_k = w$, γ_k acts as a string.

Where ' x_i ' gets eliminated from the input '*w*'

Step 1: Basis Step

Consider $K = 1$ and $\gamma_1 = Z$ thus the following is obtained

$$x_i = x_j = \epsilon \text{ and } y_i = y_j = w$$

Since,

$$x_i \alpha_k = \gamma_k$$

If $k = i$ then

$$x_i = \epsilon \text{ and } \alpha_i = \gamma_i = Z$$

Consider the equation $(q, w, Z) \xRightarrow{*}_P (q, y_i, \alpha_i)$ by substituting $y_i = w$ and $\alpha_i = Z$, the following is obtained

$$(q, w, Z) \xRightarrow{*}_P (q, w, Z) \quad \dots (2)$$

Step 2: Induction Step

Now, in the induction step, it must be proved that,

$$(q, w, Z) \xRightarrow{*}_P (q, \gamma_{k+1}, \alpha_{k+1})$$

To prove this, consider left-sentential forms and induction '*i*' i.e.,

$$(q, w, s) \xRightarrow{*}_P (q, y_i, \alpha_i)$$

Where, ' α_i ' is tail of ' γ_k ' consider a variable '*E*' as head to ' γ_k ' since in induction, ' $k+1$ ' steps are considered, thus derivation γ_k will be $\gamma_k = \gamma_{k+1} E$. Based on the production *P*, '*E*' must be replaced by its production bodies like ' β '. The construction of '*P*' in γ_k follows some rules that helps in obtaining the ID $(q, \gamma_{k+1}, \alpha_{k+1})$, which represents the next left-sentential form γ_{k+1} as follows,

(i) Rule 1

Replace terminal '*E*' with ' β ' at the top of the stack.

(ii) Rule 2

Match the terminals on top of the stack with next input symbols.

Consider $\alpha_n = \epsilon$, where ' α_n ' is the tail of ' γ_n ' which has no value (empty). By this $(q, w, s) \xRightarrow{*}_P (q, \epsilon, \epsilon)$ is formed. It is proved that PDA '*P*' accepts the input '*w*' by the empty stack i.e. α_n .

Hence based on these rules the following IDs are reached. They are,

$$(q, \gamma_{k+1}, \alpha_{k+1}) \quad \dots (3)$$

This equation denotes the next left-sentential form of γ_{k+1} .

Alternative Method

Assume that, the PDA '*P*' pops from top of its stack by creating an effects PDA '*P*' and executing its moves in sequence. If the popped variable '*A*' is again derived in grammar '*G*' in the form of any input during the process then the equation will be as follows.

$$\text{if } (q, \gamma, \alpha) \xRightarrow{*}_P (q, \gamma, \alpha) \text{ then } \dots (4)$$

Next, another proof should be shown as an induction on the number of moves which is taken by PDA '*P*' i.e., with basis and induction step.

Step 1: Basis Step

In this step, $E \rightarrow \epsilon$ is defined as the production of grammar 'G' and is used in rule belonging to type 1 by PDA 'P'. Here if,

$$x = \epsilon \text{ then } E = \epsilon$$

Hence, Basis step is proved.

Step 2: Induction Step For 'n' Moves

Take 'P' with 'n' moves, where $n \geq 1$. The first move in PDA 'P' should be in the rule of type (1). So, that variable 'E' is replaced with its productions on the top of the stack. This is because, according to rule (2) a production 'P' can include only the terminals on the top of the stack. That is, if variable 'E' includes productions like $A \rightarrow Y_1, Y_2, \dots, Y_n$, then 'Y_k' consists of terminal/variable.

For $n-1$

In the above sentence, it is proved that 'Y_k' is either terminal (or) variable for 1st move. Then for $n-1$ moves, it should be proved same, for that another variable called 'x' must be derived from the input along with the net effect of deleted productions like $Y_1, Y_2, \dots, Y_n$ at a time. The derived input 'w' can be written as $w_1, w_2, w_3, \dots, w_n$. The derived variable 'w₁' is popped out of the stack, this process continues until the production Y_n becomes the terminal and popped out of the stack.

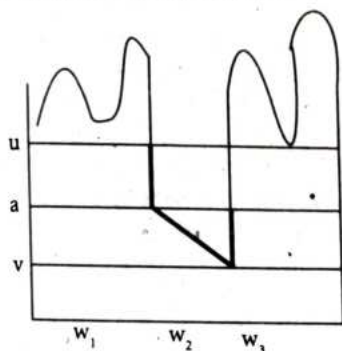


Figure: 'P' Consumes the Variable 'w' and Deletes uav from its Stack

In the above figure, it can be shown that, the variable 'w' is split into w_1, w_2, w_3 and $w_2 = a$ and also 'B' becomes uav. So, if Y_k becomes terminal, then X_k also becomes terminal.

The inductive hypothesis step is applied if 'Y_k' is a variable. It can be concluded that $(q, x_k, x_{k+1}, \dots, x_m, y_k) \xrightarrow{P} (q, x_k, x_{k+1}, \dots, x_m, \epsilon) \forall i = 1, 2, \dots, m$.

There exist only 'n-1' moves and thus $Y_k \Rightarrow x_k$

If Y_k is terminal, then only a single move is involved to the 'x_k' sequences with y_k . By this it can be concluded that, $Y_k \Rightarrow x_k \forall n = 0$.

$E \Rightarrow Y_1, Y_2, \dots, Y_m \Rightarrow x_1, y_1, y_2, \dots, y_m \Rightarrow \dots \Rightarrow x_1, x_2, x_n$ where $E \Rightarrow x$.

Now, assume $E' = Z$ and $x = w$. Since, it is given that 'w' is in $N(P)$.

i.e. $(q, w, Z) \xrightarrow{*} (q, \epsilon, \epsilon)$ from inductive step.

Then it can be proved that $Z \Rightarrow w$ where, w is in $L(G)$.

$\therefore$ From the above process it is proved that $N(P) = L(G)$.

Q13. Convert the grammar $S \rightarrow 0S1/A, A \rightarrow 1A0/S/\epsilon$ into PDA that accepts the same language by empty stack. Check whether 0101 belongs to $N(M)$

Answer :

Given grammar (G) is,

$$S \rightarrow 0S1/A$$

$$A \rightarrow 1A0/S/\epsilon$$

Let PDA be = $\{Q, \Sigma, \tau, \delta, S, F\}$

Where,

$$Q = \{p, q\}$$

$$\Sigma = \{0, 1\}$$

$$\tau = \{S, A, 0, 1\}$$

$$S = \{p\}$$

$$F = \{q\}$$

The transition function δ is given as follows,

$$R_1 : \delta(p, \epsilon, \epsilon) = \{(q, S)\}$$

$$R_2 : \delta(q, \epsilon, S) = \{(q, 0S1)\}$$

$$R_3 : \delta(q, \epsilon, S) = \{(q, A)\}$$

$$R_4 : \delta(q, \epsilon, A) = \{(q, 1A0)\}$$

$$R_5 : \delta(q, \epsilon, A) = \{(q, S)\}$$

$$R_6 : \delta(q, \epsilon, A) = \{(q, \epsilon)\}$$

$$R_7 : \delta(q, 0, 0) = \{(q, \epsilon)\}$$

$$R_8 : \delta(q, 1, 1) = \{(q, \epsilon)\}$$

Let us check whether the string $w = 0101$ is accepted by constructed PDA (or) not.

State	Input	Stack	Rules used
p	0101	ϵ	
q	0101	S	R_1
q	0101	0S1	R_2
q	101	S1	R_7
q	101	A1	R_1
q	101	1A01	R_4
q	01	A01	R_8
q	01	01	R_6
q	1	1	R_7
q	ϵ	ϵ	R_8 (Accepted)

$\therefore$ The constructed PDA accepts the language by the given grammar.

3.2 TURING MACHINES

3.2.1 Introduction to Turing Machine, Formal Description

Q14. Define turing machine. Give the model of turing machine. Explain its working with example.

OR

Design Turing machine and its transition diagram to accept the language:

$$L = \{a^n b^n \mid n \geq 1\}.$$

(Refer Only Topic: Example)

Answer :

Jan./Feb.-23(R18), Q10(b)

Turing Machine

It is simple mathematical model that performs calculations similar to any computing machine i.e., general purpose computer. A Turing Machine (TM) is a seven tuple machine (M) denoted as,

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where,

Q contains the finite set of states

Γ is the finite set of allowable tape symbols

B is a blank symbol of T

Σ (a subset of Γ not including B) is the set of input symbols.

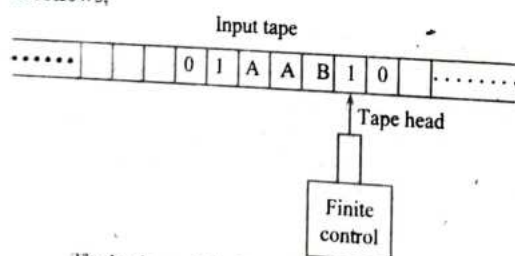
δ is the next move function, a mapping from $Q \times \Gamma$ to $Q \times \Gamma \times \{L, R\}$

q_0 is Q in the start state

$F \subseteq Q$ is the set of final states.

Model of Turing Machine

The basic model of a turing machine is represented as follows,



The basic model of the turing machine consists of,

1. Input tape
2. Finite control
3. Read/Write head tape
4. State register
5. Finite set of states and symbols.

1. Input Tape

It consists of infinite number of input cells wherein each cell contains one input symbol from some finite alphabet. The cells which are empty on tape are filled by blank characters or symbols (Δ).

2. Finite Control

It consists of a pointer that points to the current input symbol that is to be read.

3. Read/Write Tape Head

It is responsible for reading/writing input symbols on the cells of the tape and also for moving the tape left/right one cell at a time.

4. State Register

It stores the finite state of turing machine.

5. Finite Set of States and Symbols

The turing machine also consists of,

- (i) Set of states through which the machine undergo various transitions and
- (ii) Set of symbols that helps in constructing logic of turing machine.

Working of Turing Machine

The turing machine works in the following manner,

Initially, the turing machine starts its computation from a single distinguishable state called as a start with tape head pointing to the cell of the start state. The TM starts its transition by reading one input symbol from the current active cell called as start cell. It then look-up for the transition rule associated with current state and input symbol. If the turing machine encounters an input symbol 'say 0 or 1 then replaces it with a new symbol and changes its state based on the transition rule. Depending upon the new state, it then shifts the read/write tape head either left/right. This process is repeated until the halting state or accepting state is reached by the turing machine.

Example

Consider a language $L = \{a^n b^n, n \geq 1\}$

The constructed turing machine must accept a set of strings containing equal number of a's and equal number of b's. Let the turning machine 'M' be,

$$M = \{Q, \Sigma, \tau, \delta, q_0, B, F\}$$

The design of turing machine is as follows.

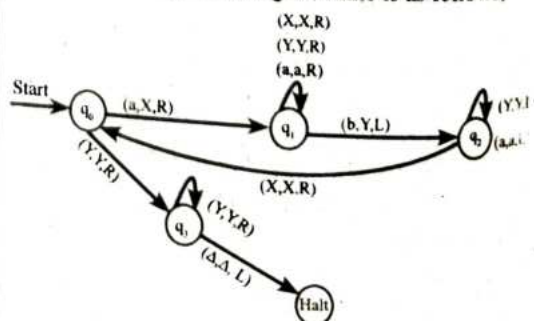


Figure: Turing Machine for $L = \{a^n b^n \mid n \geq 1\}$

The transition table for the above diagram is as follows,

	a	b	X	Y	Δ
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	(q_1, a, R)	(q_2, Y, L)	(q_1, X, R)	(q_1, Y, R)	—
q_2	(q_2, a, L)	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(Halt, Δ, L)
Halt	—	—	—	—	—

Table: Transition Table

The processing of turing machine is as follows,

Initially, 'a' takes an input 'a' and replace it by a variable 'X'.

Now, a moves the input tape to right in search of b.

If input 'b' is found, then replace it as 'Y'.

The above process is repeated until all the a's and b's are replaced by X and Y respectively.

Once the process gets completed, it enters into halting state and stops processing.

If the turing machine encounters any unequal number of a's and b's then it never halts and enter into infinite loops.

The processing of input string $w = aabb$ is as follows,

```

a  a  b  b  Δ  replace 'a' to X and move right
↑
X  a  b  b  Δ  skip 'a' and move a head to right
↑
X  a  b  b  Δ  replace 'b' by Y and move left till X is encountered
↑
X  a  Y  b  Δ  move left
↑
X  a  Y  b  Δ  move right
↑
X  a  Y  b  Δ  replace 'a' to 'X' and move right.
↑
X  X  Y  b  Δ  move right
↑
X  X  Y  b  Δ  replace 'b' to Y and move left.
↑
X  X  Y  Y  Δ  move left
↑
X  X  Y  Y  Δ  move right because, all the a's are replaced by 'X'.
↑
X  X  Y  Y  Δ  move right
↑
X  X  Y  Y  Δ  move right
↑
X  X  Y  Y  Δ  Halt
↑

```

Q15. Discuss in detail about the transition table and transition diagram of the turing machine.

Answer :

Transition Table

Transition table is used to define a transition function (δ). Generally, it is represented in the tabular form which consists of rows and columns. The rows and columns are filled such that, if $\delta(q, a) = (\gamma, \alpha, \beta)$ then α β γ is inserted into α^{th} -column and q^{th} row.

Where $\alpha \rightarrow$ It is the present cell.

$\beta \rightarrow$ It indicates the movement of the head that is either left or right

$\gamma \rightarrow$ It indicates the new state in which TM is going to enter.

Example

Consider a TM in which five states say q_0, q_1, q_2, q_3 and q_4 wherein q_0 and q_4 are the initial and final states respectively. However, the tape symbols are denoted as 0, 1, b. The transition table of TM is shown below.

Present State	Tape Symbol		
	b	0	1
$\rightarrow q_0$	1Lq ₁	0Rq ₀	—
q_1	bRq ₂	0Lq ₁	1Lq ₁
q_2	—	bRq ₃	bRq ₄
q_3	0Rq ₄	0Rq ₃	1Rq ₃
$\odot q_4$	0Lq ₁	—	—

Representation of Turing Machine by Transition Diagram

A Turing Machine (TM) is a simple mathematical model of general purpose computer. A turing machine denoted as $M = (Q, \Sigma, \tau, \delta, Q_0, B, F)$ contains a transition functions Σ of form,

$$Q \times \tau \leftarrow (Q \times \tau, \Delta (L, R))$$

In the above transition function, the L.H.S represents the current state and the RHS contains the current input that belongs to τ . To represent these transition function, graphically a generalized transition diagram is used. The transition diagram of turing machine consists of,

1. A set of nodes representing the input state. The label at the beginning of state is called a start state or initial state and a node with double circle at the end represents the final state or accepting state.
2. A pair of arrows/edges between states contain transitions. These transitions consist of input symbol, directions like left, right or halt of read/write head, the current input symbol on the input tape after traversal.

Example

The transitional representation of the language $L = \{a^n b^n / n > 0\}$ is shown below.

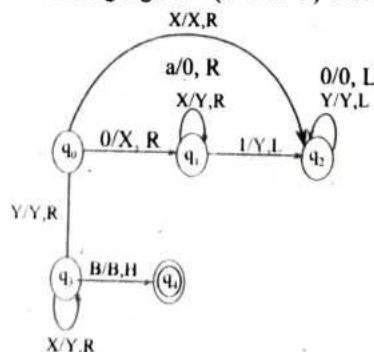


Figure: Transition Representation of Language

A 5-tuple specification is a method that describes the operation of a turing machine that is carried out from one state to another. This method makes use of five different tuples such as, State-1, Read Symbol, Write Symbol, L/R/N, State-2 in order to specify the function of TM.

Q16. Differentiate between Finite Automata and Turing Machine.

Answer :

Finite Automata (FA)	Turing Machine (TM)
1. Finite Automata is defined as a model, which helps in designing and checking the behaviour of digital circuits within the system software.	1. Turing Machine is defined as a mathematical model, which perform calculations similar to any computing machine.
2. Finite Automata can read from tape but cannot write on tape.	2. Turing Machine can read and write data onto the tape.
3. Finite Automata cannot move its read and write head to left and right.	3. Turing Machine can move its read and write head to its left and right.
4. The tape of Finite Automata is finite.	4. The tape of Turing Machine is infinite.
5. Finite Automata does not contain accepting and rejecting states.	5. Turing Machine contains two special states i.e., accepting state and rejecting state.
6. Finite Automata reads the entire input inorder to ensure its acceptance.	6. Turing Machine does not read the entire input inorder to ensure its acceptance.
7. Finite Automata halts only after reaching the final state input.	7. Turing Machine halts on acceptance of any input.
8. Finite Automata cannot halt and reject any input over regular language.	8. Turing Machine can halt and reject any input over recursive language.

3.2.2 Instantaneous Description

Q17. Discuss in detail about the instantaneous description of turing machine.

Answer :

Instantaneous Description/Configuration of a Turing Machine

An Instantaneous Description (ID)/Configuration of a Turing Machine (TM) is defined as the combination of present state and the complete input string. It is written as $\alpha_1 q \alpha_2$, where α_1, α_2 are the input strings in Σ^* and α_1 is the substring of the input string. The first symbol of α_2 is the current symbol under the Read/Write (R/W) head and following are the subsequent symbols of the input string. q is the current state of the TM, two IDs are related by the symbol $\vdash_{TM}$ and, multiple IDs are related by the symbol $\vdash_{TM}^*$, which is the reflexive transitive closure of $\vdash_{TM}$.

Moves of a Turing Machine

A move in a Turing Machine (TM) is defined as a transition from one state to another state on a tape symbol (input or initial tape symbol like blank symbol 'B'). The transition function is denoted by ' δ '. Its general form is,

$$\delta(q, i_n) = (p, j, H_d)$$

Where,

 q = Current state i_n = Input symbol p = Next state j = Symbol that replaces the current symbol H_d = Head direction Right(R), Left (L).

The transition function is read as an input ' i_n ' the machine moves from state ' q ' to state ' p ' by replacing ' i_n ' by ' j ' and moving the tape head one cell either to left or to right (depending on the head direction, H_d).

If the transition function is given as $\delta(q, i_n) = (p, j, L)$, then the ID before processing input symbol is given as,

$$(i_1 i_2 \dots i_{n-1} q i_n \dots i_m).$$

The ID after processing is given as,

$$(i_1 i_2 \dots i_{n-2} p i_{n-1} j i_{n+1} \dots i_m).$$

These two IDs are related as,

$$i_1 i_2 \dots i_{n-1} q i_n \dots i_m \xrightarrow{T/M} i_1 i_2 \dots i_n p i_{n+1} i_{n+2} \dots i_m$$

If the transition function is given as, $\delta(q, i_n) = (p, j, R)$ then, the change in ID represented as follows,

$$i_1 i_2 \dots i_{n-1} q i_n \dots i_m \xrightarrow{T/M} i_1 i_2 \dots i_n j p i_{n+1} \dots i_m$$

Example

The turing machine for recognizing the language $L = \{x \in \{a, b\}^* / x \text{ ends with } aba\}$ that is set of all strings that end with aba is shown below.

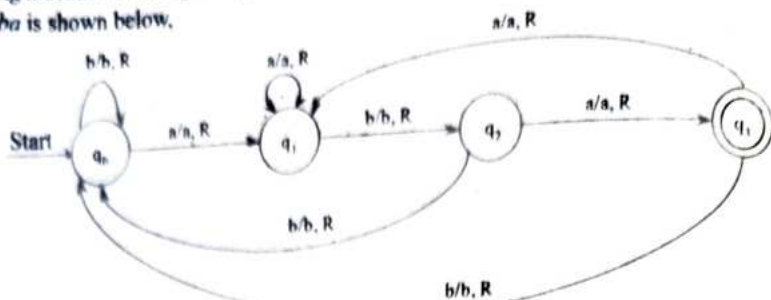


Figure: Transition Diagram

Let the turing machine be $M = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$

Where,

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_3\}$$

The transition table is given below,

States	Tape Symbols	
	a	b
q_0	(q_1, a, R)	(q_0, b, R)
q_1	(q_1, a, R)	(q_2, b, R)
q_2	(q_3, a, R)	(q_0, b, R)
q_3	(q_1, a, R)	(q_0, b, R)

Table: Transition Table

To process two strings i.e., $w_1 = abaaba$ and $w_2 = ababaa$ using instantaneous description of Turing machine.

- (1) Check the acceptance of string $w_1 = abaaba$

$\vdash q_0 abaaba$
 $\vdash^R aq_1 baaba$
 $\vdash^R abq_2 aaba$
 $\vdash^R abaq_3 aba$
 $\vdash^R abaaq_1 ba$
 $\vdash^R abaabq_2 a$
 $\vdash^R abaabaq_3$

↑
Final state

Since the final state q_3 is reached, the turing machine accepts the string $abaaba$.

(ii) Check the acceptance of string $w_2 = ababaa$

$\vdash q_0 ababaa$

$\vdash^R aq_1 habaa$

$\vdash^R abq_2 abaa$

$\vdash^R abaq_3 baa$

$\vdash^R ababq_0 aa$

$\vdash^R ababaq_1 a$

$\vdash^R aababaaq_1$

Non-final state

As the final state q_3 is not reached, and as the input string ends on the state q_1 , the input string $ababaa$ is not accepted by the turing machine.

3.2.3 The Language of a Turing Machine

Q18. Explain in detail about language of turing machine. Also give example.

Answer :

Languages Accepted by Turing Machine

Let the turing machine $M' = \{Q, \Sigma, \Gamma, \delta, q_0, B, F\}$

Where,

Q is the finite set of states

Γ is the finite set of allowable tape symbols

$B \in \Gamma$, is the blank

Σ a subset of Γ not including B , is the set of input symbols

δ is the next move function, a mapping from $Q \times \Gamma$ to $Q \times \Gamma \times \{L, R\}$

q_0 is Q in the start state

$F \subseteq Q$ is the set of final states.

Let the language accepted by M' be $L(M')$

Where, $L(M')$ represents a finite set of input string in Σ^* which enables the turing machine M' to enter from initial state (q_0) to the final state (q_f).

Symbolically, this can be represented as,

$$L(M') = \{w/w \in \Sigma^* \text{ and } q_0 W \xrightarrow{M} \alpha' q_f B' \text{ for some } q_f \text{ and } \alpha' \text{ and } B' \in \Gamma^*\}.$$

The language accepted by TM is usually called as recursively enumerable language.

A turing machine for the given language $(a + b)^* aba(a + b)^*$ is shown below.

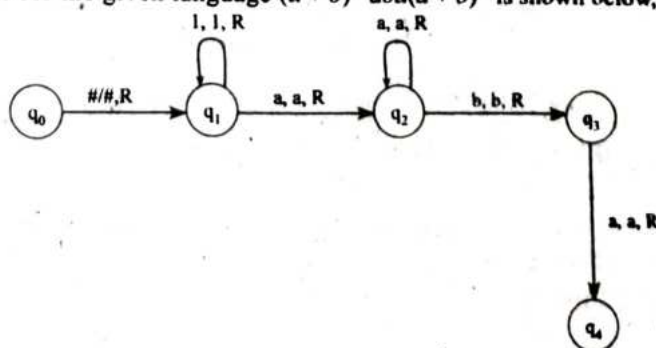


Figure: Turing Machine for Given Language $R \rightarrow \text{Right} \# \rightarrow \text{Blank Symbol}$

The above turing machine accepts a set of strings that contain *aba* as a substring. After discovering *aba* on the tape, it halts on final state q_4 by accepting the entire input string, even if all the input symbols are not read.

Example

Let us check the acceptance of the string

$$W = aabaabaaab$$

$$q_0 \# aabaabaaab$$

R	$q_1 aabaabaaab$
R	$aq_2 abaabaaab$
R	$aaq_3 baabaaab$
R	$aabq_4 aabaaab$
R	$aabaq_5 aabaaab$

As the substring *aba* is encountered by the input tape and as the final state is reached, the turing machine (M) halts and accepts the entire input string.

Q19. Design a turing machine to accept $L = \{wCw^R \mid w \text{ is in } (a+b)^*\}$.

(May-19(R16), Q9 | May-17(R13), Q8(a))

OR

Design a turing machine to accept the language $L = \{wCw^R \mid w \in (a+b)^*\}$.

Dec.-18(R16), Q8(a)

OR

Design a TM to accept the language $L = \{wCw^R \mid w \in (a+b)^*\}$.

Answer :

Dec.-17(R13), Q9(a)

The given language is,

$$L = \{wCw^R \mid w \text{ is in } (a+b)^*\}$$

Where w^R is reverse of a string i.e., if,

$$w = aab \text{ then } w^R = baa.$$

The string accepted by language L will be,

$$w = aabCbaa$$

Let the turing machine be,

$$M = \{Q, \Sigma, \delta, \Gamma, q_0, F\}$$

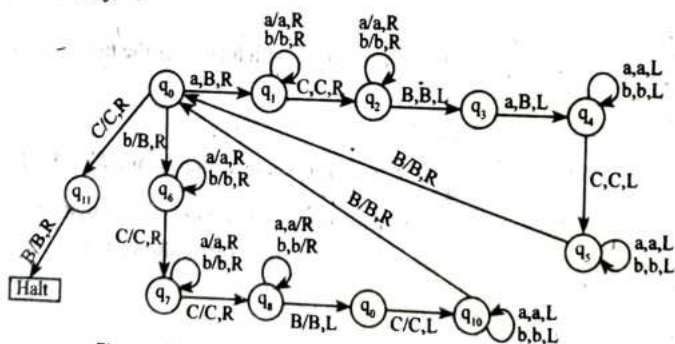


Figure: Turing Machine for $L = \{wCw^R \mid w \in (a+b)^*\}$

Q20. Design a TM to recognize the language $L = \{1^n 2^n 3^n \mid n \geq 1\}$.

May-17(R13), Q8(b)

OR

Construct Turing Machine to accept following language and give its state Transition table and diagram. Check the machine by tracing a suitable instance.

$$L = \{a^n b^n c^n \mid n \geq 1\}. \text{ [Consider } a, b, c \text{ as } 1, 2, 3]$$

Answer :

Given language is $L = \{1^n 2^n 3^n \mid n \geq 1\}$

Sep.-21(R18), Q8(a)

Let the turing machine be,

$$M = (Q, \Sigma, \Gamma, \delta, q_0, b, f)$$

Where,

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$$

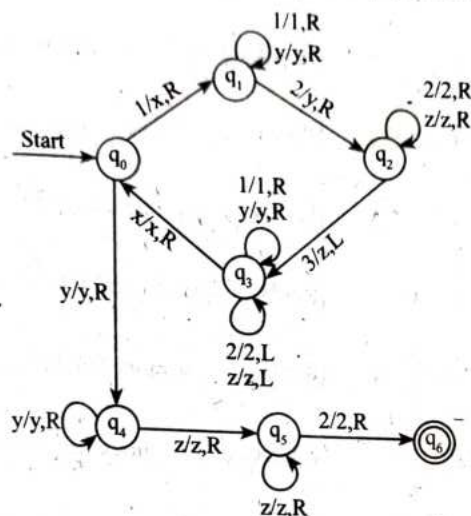
$$\Sigma = \{1, 2, 3\}$$

$$\Gamma = \{a, b, c, x, y, z, b\}$$

$$q_0 = q_0$$

$$f = q_6$$

The transition diagram to recognize the language $L = \{1^n 2^n 3^n \mid n \geq 1\}$ is as follows,



The transition table is as follows,

States	Tape Symbols						
	1	2	3	x	y	z	b
→ q ₀	(q ₁ , x, R)	—	—	—	(q ₄ , y, R)	—	—
q ₁	(q ₁ , 1, R)	(q ₂ , y, R)	—	—	(q ₁ , y, R)	—	—
q ₂	—	(q ₂ , z, R)	(q ₃ , z, R)	—	—	(q ₂ , z, R)	—
q ₃	(q ₃ , 1, L)	(q ₃ , 2, L)	—	(q ₀ , x, R)	(q ₃ , y, L)	(q ₃ , z, L)	—
q ₄	—	—	—	—	(q ₄ , y, R)	(q ₅ , z, R)	—
q ₅	—	(q ₆ , 2, R)	—	—	—	(q ₅ , z, R)	—
q ₆	—	—	—	—	—	—	—

The processing of turing machine is as follows,

Initially it takes an input 1 and replace it by a variable 'x'.

Now, 1 moves the input tape to right in search of 2. If 2 is found it is replaced with variable 'y'.

Now, 2 moves the input tape to right in search of 3. If 3 is found it is replaced with variable 'z'.

The above process is repeated until 1's, 2's and 3's are replaced by x, y and z respectively.

Once the process gets completed, it enters into halt state after which the processing stops.

If the turing machine encounters unequal number of 1's, 2's and 3's it enters into infinite loop.

The processing of input string $w = 11122233$ is as follows.

1	1	1	2	2	2	3	3	3	Δ	[Replace 1 by 'x' and move right]
↑										
x	1	1	2	2	2	3	3	3	Δ	[Skip it and move right to 2]
↑										
x	1	1	2	2	2	3	3	3	Δ	[Replace 1 by 'y' and move right]
			↑							
x	1	1	y	2	2	3	3	3	Δ	[Skip it and move ahead to right 3]
			↑							
x	1	1	y	2	2	3	3	3	Δ	[Replace 3 by 'z', move left till x is encountered and then move right]
					↑					
x	1	1	y	2	2	z	3	3	Δ	[Replace 1 by 'x', move right till y is encountered and move right]
↑										
x	x	1	y	2	2	z	3	3	Δ	[Replace 2 by 'y', move right till z is encountered and move right]
			↑							
x	x	1	y	y	2	z	3	3	Δ	[Replace 3 by z, move left till x is encountered and then move left]
					↑					
x	x	1	y	y	2	z	z	3	Δ	[Replace 1 by x, move left till y is encountered and then move right]
↑										
x	x	x	y	y	2	z	z	3	Δ	[Replace 2 by y, move left till z is encountered and then move right]
				↑						
x	x	x	y	y	y	z	z	3	Δ	[Replace 3 by z, move right till Δ is encountered]
							↑			
x	x	x	y	y	y	z	z	z	Δ	
								↑		
									Halt	

Since all 1's, 2's and 3's are replaced by x, y, z respectively. Thus, the above constructed turing machine accepts the language $L = \{1^n 2^n 3^n \mid n \geq 1\}$.

Q21. Design a turing machine over $\Sigma = \{a, b\}$ to accept the language $L = \{ww^R \mid w \in (a, b)^*\}$.

Dec.-19(R16), Q3

OR

Design a turing machine to accept $L = \{ww^R \mid w \text{ is in } (a + b)^*\}$.

Answer :

Given language is,

$$L = \{WW^R \mid w \in (a, b)^*\}$$

The given language accepts set of strings containing palindrome.

Thus the language L will be,

$$L = \{aa, bb, abba, baab, \dots\}$$

A turing machine to recognize the language of palindrome over $\{a, b\}$ can be constructed by defining moves in the following manner,

- The machine scans the first input symbol (either 0 or 1), erases (but remembers) it, writes a blank symbol at its place and changes its state (q_1 or q_2).
- It scans the remaining part without changing the tape symbol until it encounters b . It then moves the read/write head a step left. If the rightmost symbol tallies with the leftmost symbol, the rightmost symbol is erased. Otherwise TM halts. The read/write head moves to the left until b is encountered.
- The above steps are repeated after changing the states suitably.

WARNING: Xerox/Photocopying of this book is a CRIMINAL act. Anyone found guilty is LIABLE to face LEGAL proceedings.

May-17(R13), Q3(a)

The transition table is shown below,

Present States	Tape symbols		
	a	b	B
$\rightarrow q_0$	BR q_1	BR q_4	BR q_6
q_1	aR q_1	bR q_1	BL q_2
q_2	BL q_3		BR q_6
q_3	aL q_3	bL q_3	BR q_0
q_4	aR q_4	bR q_4	BL q_5
q_5		BL q_3	BR q_6

Table: Transition Table

The transition diagram is shown in figure below,

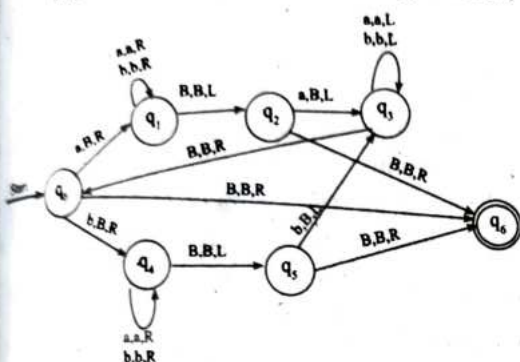


Figure: Turing Machine for Palindromes

1. Given string is *abab*

The moves on Turing Machine of the string *abab* is as follows,

From above transition table,

$q_0 abab \vdash Bq_1 bab$
 $\vdash Bbq_1 ab$
 $\vdash Bbaq_1 b$
 $\vdash Bbabq_1$
 $\vdash Bbabq_1 B$ (Assume B as blank state)
 $\vdash Bbaq_1 bB$

Since, there is no transition for q_1 on *b*, the machine does not reach the final state.

The given string *abab* is not accepted by the turing machine.

2. Given string is *baab*

The moves on Turing Machine of the string *baab* is as follows,

From above transition table,

$q_0 baab \vdash Bq_4 aab$
 $\vdash Baq_4 ab$
 $\vdash Baaq_4 b$
 $\vdash Baabq_4$
 $\vdash Baabq_4 B$ (Assume B as blank state)

$\vdash Baaq_5 b$
 $\vdash Baq_5 aB$
 $\vdash Bq_5 aaB$
 $\vdash q_5 BaaB$
 $\vdash Bq_5 aaB$
 $\vdash BBq_5 aB$
 $\vdash BBaq_5 B$
 $\vdash BBq_5 aB$
 $\vdash Bq_5 BBB$
 $\vdash BBq_5 BB$
 $\vdash BBBq_5 B$

Since, the final state q_6 is reached, the string $W = baab$ is accepted by the turing machine.

Q22. Consider the following Turing Machine $M = (\{q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, B\}, \delta, q_1, B, q_4)$ where δ is given as,

$\delta(q_1, 0) = (q_2, X, R)$

$\delta(q_2, 0) = (q_2, X, R)$

$\delta(q_2, 1) = (q_3, X, R)$

$\delta(q_3, 0) = (q_2, X, R)$

$\delta(q_3, 1) = (q_3, X, R)$

$\delta(q_3, B) = (q_4, X, R)$

What will be the initial and final configurations of the Turing machine for the input string $w = 0101$?

Answer :

Let the Turing Machine $M = (\{q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, B\}, \delta, q_1, B, q_4)$ where δ is given as,

$\delta(q_1, 0) = (q_2, X, R)$

$\delta(q_2, 0) = (q_2, X, R)$

$\delta(q_2, 1) = (q_3, X, R)$

$\delta(q_3, 0) = (q_2, X, R)$

$\delta(q_3, 1) = (q_3, X, R)$

$\delta(q_3, B) = (q_4, X, R)$

A turing Machine that recognizes input string $w = 0101$ is shown in figure below,

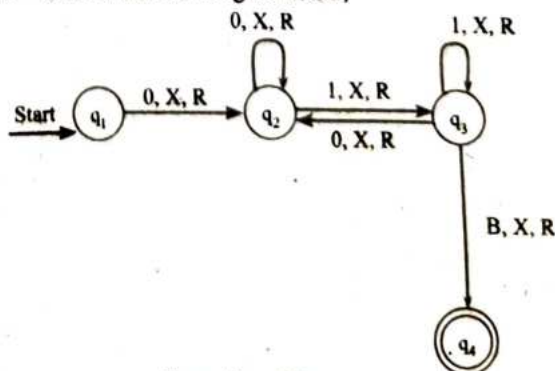
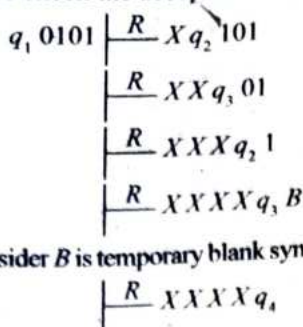


Figure: Transition Diagram

Let us check the acceptance of string $w = 0101$,



(Consider B is temporary blank symbol in right end)

Since final state q_4 is reached, the turing machine accepts the string $w = 0101$.

Initial Configuration is $q_1, 0101$

Final Configuration is $X X X X q_4$.

Q23. Construct Turing machine for the languages containing the set of all strings of balanced parenthesis.

Answer :

Jan./Feb.-23(R18), Q10(a)

Consider the turing machine,

$$M = (Q, \Sigma, \Gamma, \delta, q_0, b, f)$$

Where,

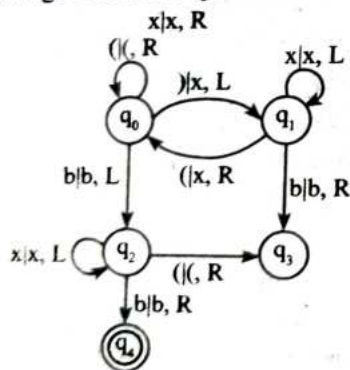
$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{ (,) \}$$

$$\Gamma = \{ [,], x, b \}$$

$$f = \{q_4\}$$

The turing machine for the languages containing set of all strings of balanced parenthesis is as follows,



The transition table for the given language is as follows,

	(	)	x	b
$\rightarrow q_0$	$q_0, (, R$	q_1, x, L	q_0, x, R	q_2, b, L
q_1	q_0, x, R	—	q_1, x, L	q_3, b, R
q_2	$q_3, (, R$	—	q_2, x, L	q_4, b, R
q_3	—	—	—	—
q_4	—	—	—	—

3.3 UNDECIDABILITY

Q24. What is decidability? Explain in brief about any two undecidable problems.

Jan./Feb.-23(R18), Q11(b)

OR

Explain about the decidability and undecidability problems.

April-18(R16), Q10(b)

OR

Discuss the examples of undecidable problems.

May-19(R16), Q10(b)

(Refer Only Topic: Unsolvability/Undecidability Problems)

OR

What are undecidable problems? Give examples.

(Refer Only Topic: Unsolvability/Undecidability Problems)

Answer :

Dec.-17(R13), Q10(b)

Decision Problem

In computability theory, a decision problem is defined as a problem which produces its results in the form of yes (or) no to the given input values.

Types of Decision Problems

Decision problems are of two types,

1. Solvable/decidability problems
2. Unsolvability/undecidability problems.

1. Solvable/Decidability Problems

A problem is said to be solvable if its supportive language is "recursive". These problems are easily solved by using algorithmic approach. Here the word "decidable" refers to the word "recursive". Hence, it can be said that for a given set of problems if there exists an algorithm which can successfully produce two outputs namely "Yes" (or) "No", then such set of problems are said to be set of decidable problems.

Examples

The following are the list of decidable problems with their answer as Yes (or) No.

- (i) Sun rises in the east ?
Answer : Yes
- (ii) Earth moves around the moon ?
Answer : No

The following are the examples for decidability problems with respect to computation theory,

Decidable Problems for Finite Automata

- (a) For a given Finite Automata 'M' and input alphabet Σ .
- ❖ $L(M) = \phi$ is decidable.
 - ❖ $w \in L(M)$, where w is a string in Σ is decidable.
 - ❖ $L(M) = \Sigma^*$ is decidable.
- (ii) For the given two finite automata's M_1 and M_2 and input alphabet Σ , $L(M_1) = L(M_2)$ is decidable.

Decidable Problems for Recursive and Recursively Enumerable Languages

A recursively enumerable language is decidable if it is recursive.

- (i) For a given recursively enumerable language L , L' is recursive.
- (ii) For the given two recursive languages L_1 and L_2 ,
- ❖ $L_1 \cup L_2$ is recursive and
 - ❖ L_1' is recursive (or) L_2' is recursive.

Decidable Problems for CFG's and CFL's

- (i) For the given two CFL's L_1 and L_2 , it is decidable that $L_1 \cup L_2$ is CFL.
- (ii) For the given two CFL's L_1 and L_2 , it is decidable that $L_1 L_2$ is CFL.
- (iii) For a given CFL L , it is decidable that L^* is CFL.
- (iv) For a given regular language L_1 and a CFL L_2 , it is said to be decidable that $L_1 \cup L_2$ is CFL.
- (v) For a given CFG, 'G',
- ❖ $L(G) = \phi$ is decidable
 - ❖ $L(G)$ is finite (or) infinite is decidable.

2. Unsolvability / Undecidability Problems

A problem is said to be unsolvable if its corresponding language is non-recursive. In other words, an unsolvable problem has no solution (or) answer (or) algorithm. Hence, finally it can be said that for a given set of problems, if there exists no solution (or) algorithm which cannot successfully produce two outputs (i.e., yes (or) no), then such set of problems are said to be the set of undecidable (or) unsolvable problems.

Examples

- (a) Post Correspondence Problem (PCP).
- (b) Halting Problem.

(a) Post Correspondence Problem (PCP)

Post Correspondence Problem (PCP) is defined as an undecidability problem over an alphabet ' Σ ' which is stated as follows,

Let X and Y are the two lists of non-empty strings over Σ such that,

$$X = (X_1, X_2, X_3, \dots, X_n)$$

$$Y = (Y_1, Y_2, Y_3, \dots, Y_n)$$

It can be said that, there is a post correspondence solution, if for some $i_1, i_2, \dots, i_k$, where $1 \leq i_k \leq n$, the condition $X_{i_1} \dots X_{i_k} = Y_{i_1} \dots Y_{i_k}$ satisfies.

(b) Halting Problem

Halting problem is defined as a decision problem, which determines whether a given program produces an output (or) not (halt) for a given input. It is also a best example for showing undecidability of a language over an alphabet ' Σ '.

3.3.1 A Language that is Not Recursively Enumerable

Q25. Discuss in detail about recursively enumerable languages. Prove that diagonalization language is not recursively enumerable.

OR

Give an overview of recursively enumerable language.

Answer : (Feb.-22(R18), Q8(a) | April-18(R16), Q11(a))

Recursively Enumerable Language

A language L over the alphabet Σ is called Recursively Enumerable (RE) if there is a TM M that accepts every word in L and either rejects (crashes) or loops for every word in language L' i.e., the complement of L .

$$\text{accept}(M) = L$$

$$\text{reject}(M) + \text{Loop}(M) = L'$$

When TM M is still running on some input (of recursively enumerable languages), it can never determine whether M will eventually accept if it run for long time or M will run forever (in loop).

To prove that a language comprising of pairs (M, w) is not RE, consider that,

M : Corresponds to a turing machine with input alphabet $\{0, 1\}$

w : Corresponds to a binary string containing zero's and one's and w is accepted by M .

Diagonalization Language (L_d)

Diagonalization Language (L_d) is a language which includes the set of strings ' w_i ' that are not accepted by the language of turing machine ' T_i ' i.e., $L(T_i)$. It can be represented as follows,

$$L_d = \{ w_i \mid w_i \notin L(T_i) \}$$

Here,

L_d = Diagonalization Language

w_i = Set of strings

T_i = Turing machine.

Diagonalization language makes use of diagonalization table in order to check the acceptance of a string by a turing machine T_i . Let 'i' indicate rows of the table and 'j' indicate columns in the table. The variable 'i' and 'j' are filled with inputs 0's and 1's respectively. Here, '0' indicates string rejected by the turing machine (T_i) and '1' indicates string accepted by the turing machine (T_i). The i^{th} row represents the characteristic vector for the language $L(T_i)$. This means that, 1's in the row represents the string which are members of the language $L(T_i)$. The diagonalization table is depicted below.

	1	2	3	4	-----	
1	1	0	1	1	0	-----
2	1	1	1	0	0	-----
3	0	0	1	1	-----	
4	0	1	0	1	-----	
⋮	⋮	⋮	⋮	⋮	⋮	-----

Figure (1): Diagonalization Table

In the above table, the values present in the diagonal are used for determining whether T_i accepts w_i or not. The construction of L_d is carried out by complementing the diagonal. The below figure represents the complemented diagonals which are of the form 1, 0, 0, 0,

	1	2	3	4	-----	
1	1	0	1	1	0	-----
2	1	1	1	0	0	-----
3	0	0	1	1	-----	
4	0	1	0	1	-----	
	1	1	1	1	1	-----

Complemented diagonal i.e., 1_{ij}

Figure (2): Diagonalization Table with Complemented Diagonals

Therefore, L_d contains $w_1 = \epsilon$ but does not contain w_2 through w_4 that are 0, 1 and 00 etc.

The technique of complementing the diagonal is used for constructing characteristic vector of a language. The characteristic vector of a language is a language which is not present in any row. This technique is known as "Diagonalization". The complement of a diagonal is itself a characteristic vector that illustrates the memberships in language L_d . Sometimes this characteristic vector differs in few columns with respect to individual rows of the table. Hence, the complement of diagonal may not be in the characteristic vector in any of the turing machine (T_i).

Theorem

L_d is not recursively enumerable language. This means that, there exist no turing machine (T) which accepts L_d .

Proof

Let us consider L_d as a diagonalization language over input alphabet $\{0,1\}$. Also assume that, $L(T)$ is a language of some turing machine T . This turing machine T is already constructed using input alphabet $\{0,1\}$.

Hence, it can be said that there exist atleast a single code (i) for designing turing machine (T) such that $T = T_i$.

The algorithm which is used to check whether w_i is in L_d or not is as follows,

1. When w_i is in L_d then T_i accepts w_i . But based on the definition of L_d it can be said that w_i is not accepted by L_d since it includes only those strings (w_j) that are rejected by T_i . Thus, it can be concluded that w_i is not accepted by T_i .
2. Moreover, if ' w_i ' is not included in L_d then it gets rejected by T_i . Therefore, from definition of L_d it can be said that ' w_i ' is present in L_d . Since the acceptance of w_i by L_d is undecidable, it can be concluded that there exist no turing machine (T) for a language ' L_d '.

As no turing machine (T) exists for L_d it can be said that L_d is not recursively enumerable.

3.3.2 An Undecidable Problem that is RE

Q26. Explain about recursive language with suitable examples. Also give the differences between recursive language and recursive enumerable language.

Answer :

Recursive Language

A recursive language is a language defined over an input alphabet ' Σ ' in which a turing machine performs the following actions,

- (i) **Accepts the Input**

The input is accepted if it is present in the language, and halts in final state.

Rejects the Input

The input is rejected if it is not present in the language and halts in non-final state.

A turing machine T defines an algorithm in order to check whether a language L is recursive or non-recursive. If a language is recursive then it is called as decidable otherwise it is undecidable. Thus, it can be said that a recursive language is decidable. Hence, by dividing a problem or language into decidable (or) undecidable, a clear distinction between recursive language and non-recursive language can be obtained.

Example: $L_1 = \{a^n b^n c^n \mid n \geq 0\}$, $L_2 = \{0^n 1^n 2^n \mid n \geq 0\}$

$L_3 = L_1 \cdot L_2 = \{a^n b^n c^n 0^n 1^n 2^n \mid n \geq 0\}$

where the sets are

$L_1 = \{abc, aabbcc, \dots\}$

$L_2 = \{012, 001122, \dots\}$

$L_3 = \{abc 012, aabbcc 001122, \dots\}$.

Differences between Recursive and Recursively Enumerable Languages

Recursive Language	Recursively Enumerable Language
1. A language L over the alphabet 'Σ' is called recursive if there is a turing machine (T), that accepts every word in L, and rejects every word which is not in L. 2. It has a turing machine (T) which halts in final and non-final states. 3. It has a total acceptor. 4. It is also called as a turing machine decidable language. 5. All recursive languages are also recursively enumerable. 6. They are not recognized by the turing machine. 7. A language is recursive if and only if there exists an inventory method for it which gives weak result at the end. 8. A language is recursive if there exist a membership algorithm for it.	1. A language L over the alphabet 'Σ' is called recursively enumerable if there is a turing machine (T), that accepts every word in L, and either rejects or loops for every word in language L' (the complement of L). 2. It has a turing machine (T) which halts only in final state. 3. It has a partial acceptor. 4. It is also called as a turing machine acceptable language. 5. All recursively enumerable languages are not recursive. 6. They are recognized by the turing machine. 7. A language is said to be recursively enumerable if and only if there exist an inventory method for it which gives a strong result at the end. 8. A language is recursively enumerable as it does not exist a membership algorithm for it.

Q27. Prove that the complement of any recursive language is recursive.

Answer :

Consider a recursive language L such that there exists a turing machine (T) that can accept L . Here, turing machine provides assurance that it can either accept a string and halt (or) reject a string and then halts if $w \in L$ (or) $w \notin L$ respectively.

Now, a turing machine (T') is constructed such that it simulates T on ' w ' as shown below,

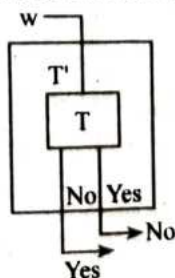


Figure: Complement of two Recursive Language.

Here, a string w is taken as input by the turing machine, since T' is the complement of T . Hence if T accepts w then T' rejects it and if T' rejects a string w , then T also rejects it. Since T' accepts the complement of T , thus it can be said that complement of any recursive language is recursive.

3.3.3 Undecidable Problems about Turing Machines

Q29. Explain about the undecidable problems regarding turing machine.

Answer :

The undecidable problems regarding the turing machine are as follows,

Reductions

Reduction is a process in which the problem X is reduced to problem Y such that the instances of X can be converted to instances of Y having same solution using an algorithm. Therefore, if X is non-recursively enumerable then Y is also non-recursively enumerable and if X is non-recursive, then Y is also non-recursive.

Reduction transforms the instances which are positive to positive and negative instances to negative. This is illustrated in the figure below,



Figure: Reduction

In the above figure, the instance of problem X whose solution is '1' is transformed into the instance of Y with same solution and the instance of problem X that has '0' is transformed into the instance of Y with the same answer (i.e., '0'). The concept of reduction helps in proving the following theorem.

Theorem

If the problem X is reduced to Y then the following are true.

- (i) If X is undecidable then Y is also undecidable.
- (ii) If X is non-RE then Y is also non-RE.

Proof

- (i) To prove that if X is undecidable then Y is also undecidable, let the problem X is undecidable. Now, design an algorithm that converts the instance of X to the instance of Y . For example, let, ' p ' be the instance of X and the algorithm converts it to ' q ' (the instance of Y). Now verify that whether ' q ' exists in the problem Y or not. If the algorithm results '1' then both X as well as Y are decidable. If the algorithm results '0' then problem Y is undecidable and as result, the problem X is also undecidable (Since ' X ' doesn't have the instance of ' Y ').
- (ii) To prove that if X is non-RE then Y is also non-RE, consider that the problem X is non-RE and ' Y is RE. Now, the algorithm that converts/reduces the problem X to Y is considered. In this case then the process considers only problem Y . This means that if the given input is in Y and the condition is satisfied, then only the turing machine returns the output. If the input is not in Y , then the Y may continuously check for the input. Therefore a specific algorithm can be used to convert the instance of X to the instance of Y . Using the instance p , it is converted to ' q '. Now, if the instance p is in language ' X ' then it is also in ' Y ', else, if it is not in the language ' Y ', then it is not in ' Y '. Hence, if ' X ' is non-RE then ' Y ' is also non-RE.

Turing Machines that Accepts the Empty Languages

Turing machines that accept empty languages can be illustrated by considering two languages i.e., empty languages denoted by ' L_e ' and non-empty languages denoted by ' L_{ne} '. These languages carry binary strings. If these binary strings are represented with ' w ', then the strings represent turing machine (M) that doesn't accepts input (i.e., $L(M) = \phi$). Therefore, L_e is said to be empty language. If the language of the turing machine $L(M)$ is a non empty language, then the string ' w ' is said to be in non-empty language (L_{ne}). The two languages can be represented as,

1. $L_e = \{M \mid L(M) = \phi\}$
2. $L_{ne} = \{M \mid L(M) \neq \phi\}$

Among these two languages, L_e is non-recursively enumerable but L_{ne} is recursively enumerable.

Theorem

L_{nc} is recursively enumerable.

Proof

This can be proved by constructing a non-deterministic turing machine which accepts L_{nc} . Let M be the turing machine and 'w' be the binary string. Then the TM is constructed as follows.

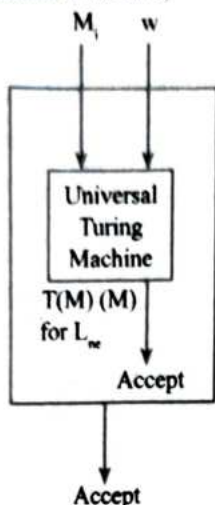


Figure: Construction of a Non-deterministic Turing Machine to Accept L_{nc}

The various operations performed by the turing machine M are as follows,

1. Turing machine (M) accepts the input as M_i which is a code of turing machine.
2. M can guess the input that can be accepted by M_i .
3. Turing machine verifies whether M_i accepts w or not.
4. If M_i accepts w , then automatically M accepts M_i (it's own input).

Here, if the M_i accepts atleast a single input string, then the turing machine can guess the string and accept M_i . Suppose, if the M_i doesn't accept any language i.e., $L(M_i) = \phi$ then, the turing machine cannot perform any guesses about the input string. Hence, M_i doesn't accept the input string. Therefore, M doesn't accept M_i . From this, it can be concluded that the inputs can be accepted only when the language of turing machine is non-empty.

$$\text{i.e., } L(M) = L_{nc} (L(M) = \phi)$$

Theorem

L_{nc} is not recursive.

Proof

The procedure followed to prove L_{nc} is not recursive is same as that of proving L_{nc} is recursive. Therefore, first an algorithm is used that converts the binary-coded pair (M, w) into a turing machine M' , such that the language of turing machine is non-empty ($L(M') = \phi$).

This theorem can be proved by constructing the turing machine M' as follows,

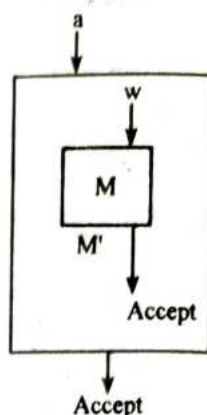


Figure: Construction of M' from the Binary-coded Pair (M, w)

Initially, the binary coded pair (M, w) is sent to the turing machine M' . If the turing machine M accepts the input string w , then automatically, M' accepts the arbitrary input that is located on the tape. According to the definition of acceptance by a TM, it accepts the input that is placed at the tape before committing to any operation. Therefore, if M accepts 'w', then the TM M' automatically accepts 'x' which is the input of M' itself. If M does not accept 'w' then the TM M' does not accept the input. In such case, the TM M' is not in L_{nc} . It can be proved by converting/ reducing the L_{nc} to L_{nc} using the algorithm which turns M' from the binary-coded pair (M, w) . Since, the L_{nc} is not recursive then L_{nc} is also not recursive.

VERY SHORT QUESTIONS WITH SOLUTIONS (VSQS)

Q1. Define pushdown automaton.

Answer :

A Push Down Automata (PDA) is a special type of automata which is designed to recognize Context Free Languages.

Q2. List the different ways of language acceptance by a PDA.

Answer :

A PDA accepts a language in two ways,

- (i) Acceptance by final state.
- (ii) Acceptance by empty stack.

Q3. Define turing machine.

Answer :

A Turing Machine is a simple mathematical model that perform calculations similar to any computing machine i.e., general purpose computer. A Turing Machine (TM) is a seven turing machine (M) is a seven tuple machine (M) denoted as,

$$M = (Q, \Sigma, \Gamma, \delta, B, q_0, F)$$

Q4. What is the purpose of studying Turing Machine?

Answer :

The purpose of studying turing machine is as follows,

1. The TM is an abstract computing machine with the power of both real computers and of other mathematical definitions of what can be computed.
2. Storage in the finite control helps to design a TM for a particular language.

Q5. State any two advantages of turing machine over other automata.

Answer :

The advantages of Turing Machine over other automata are as follows,

1. A Turing Machine has infinite tapes, while a PDA and FA has a finite tape.
2. A Turing Machine halts on any input but a PDA and FA halts only after reaching the final state.

Q6. List any two applications of turing machine.

Answer :

The following are the applications of a Turing Machine,

1. A Turing Machine is used to check the decidability of a problem.
2. A Turing Machine is used to classify the problems in polynomial hierarchy.

Q7. What do you mean by Instantaneous Description of Turing Machine?

Answer :

Jan./Feb.-23(R18), Q1(i)

An Instantaneous Description (ID)/Configuration of a Turing Machine (TM) is defined as the combination of present state and the complete input string. It is written as $\alpha_1 q \alpha_2$, where α_1, α_2 are the input strings in Σ^* and α_1 is the substring of the input string. The first symbol of α_2 is the current symbol under the Read/Write (R/W) head and following are the subsequent symbols of the input string. q is the current state of the TM, two IDs are related by the symbol ' $\vdash_{TM}$ ' and, multiple IDs are related by the symbol ' $\vdash_{TM}^*$ ', which is the reflexive transitive closure of ' $\vdash_{TM}$ '.

Q8. What is undecidable problem?

Answer :

A problem P which cannot be solved by any algorithmic approach is referred as undecidable. In other words, problems for which no algorithm exists are undecidable.

Q9. What is recursive language?

Answer :

A recursive language is a language defined over input alphabet Σ in which a turing machine performs the following actions,

1. Accepts the input if it is present in the language and halts in final state.
2. Rejects the input if it is not present in the language and halts in non-final state.

FREQUENTLY ASKED QUESTIONS & IMPORTANT QUESTIONS

Q1. Define Push Down Automata. Explain the basic structure of PDA with a neat graphical representation.

Answer :

(Important Question | Jan./Feb.-23(R18), Q6(a))

For answer refer Unit-III, Page No. 66, Q1.

Q2. Construct a PDA that accepts the language, $L \{ w c w^R \mid w \in (a + b)^* \}$

Answer :

(Important Question | Dec.-19(R18), Q6)

For answer refer Unit-III, Page No. 69, Q6.

Q3. Construct a PDA to accept the following language $L = \{ a^n b^n \mid n > 0 \}$.

Answer :

(Important Question | Sep.-21(R18), Q6(a))

For answer refer Unit-III, Page No. 73, Q11.

**Q4. Design Turing machine and its transition diagram to accept the language:
 $L = \{ a^n b^n \mid n > 1 \}$.**

Answer :

(Important Question | Jan./Feb.-23(R18), Q10(b))

For answer refer Unit-III, Page No. 76, Q14.

Q5. Explain in detail about language of turing machine. Also give example.

Answer :

Important Question

For answer refer Unit-III, Page No. 81, Q18.

Q6. Design a turing machine to accept $L = \{ w C w^R \mid w \text{ is in } (a + b)^* \}$.

Answer :

(May-19(R16), Q9 | May-17(R13), Q8(a) | Dec.-18(R16), Q8(a) | Dec.-17(R13), Q9(a))

For answer refer Unit-III, Page No. 82, Q19.

Q7. Design a TM to recognize the language $L = \{ 1^n 2^n 3^n \mid n \geq 1 \}$.

Answer :

(Sep.-21(R18), Q8(a) | May-17(R13), Q8(b))

For answer refer Unit-III, Page No. 82, Q20.

Q8. Design a turing machine over $\Sigma = \{ a, b \}$ to accept the language $L = \{ w w^R \mid w \in (a, b)^* \}$.

Answer :

(Dec.-19(R16), Q8 | May-17(R13), Q8(a))

For answer refer Unit-III, Page No. 84, Q21.

Q9. Construct Turing machine for the languages containing the set of all strings of balanced paranthesis.

Answer :

(Important Question | Jan./Feb.-23(R18), Q10(a))

For answer refer Unit-III, Page No. 86, Q23.

Q10. What is decidability? Explain in brief about any two undecidable problems.

Answer :

(Jan./Feb.-23(R18), Q11(b) | May-19(R16), Q10(b) | April-18(R16), Q10(b) | Dec.-17(R13), Q10(b))

For answer refer Unit-III, Page No. 86, Q24.

Q11. Discuss in detail about recursively enumerable languages. Prove that diagonalization language is not recursively enumerable.

Answer :

(Feb.-22(R18), Q8(a) | April-18(R16), Q11(a))

For answer refer Unit-III, Page No. 87, Q25.

Q12. Explain about the undecidable problems regarding turing machine.

Answer :

Important Question

For answer refer Unit-III, Page No. 91, Q29.